

Closure & Clones: Post’s Lattice for Bool

Closure operators provide the language for describing when a family of Boolean functions is stable under composition. In the Boolean setting, the ambient set is $\text{Bool} = \{0, 1\}$, and the central objects are sets of operations $F \subseteq \bigcup_{n \geq 1} \text{Bool}^{\text{Bool}^n}$ that are closed under superposition and contain the projections. The projections are the basic coordinate maps $\pi_i^n(x_1, \dots, x_n) = x_i$, and they serve as the identity-like generators for composition. A family with these properties is called a *clone*. The closure of a set of Boolean operations is therefore the least clone containing it, obtained by repeatedly adjoining all functions obtainable by composition with projections and previously generated operations.

A closure operator on a collection of Boolean operations is typically understood as a map Cl satisfying extensivity, monotonicity, and idempotence. In this chapter, closure is used in the algebraic sense: given a set of Boolean functions, one forms the smallest clone containing it by repeated superposition. This closure process organizes Boolean function theory into a lattice of classes ordered by inclusion. The lattice order is inclusion of clones, and the closure of a set is the least clone containing it. In the Boolean case, this lattice is completely classified by Post’s theorem.

Post’s lattice is the classification of all clones of Boolean functions. It is a finite, highly structured lattice whose nodes are the closed classes of Boolean operations on Bool . Among the most important named clones are the monotone clone M , the affine clone L , the self-dual clone D , and the classes preserving constants or distinguished values, often denoted by T_0 and T_1 . These classes capture familiar semantic constraints: monotonicity, linearity over $\mathbb{F}_2$, self-duality, and preservation of 0 or 1. In practice, they provide the first line of attack for deciding whether a given basis can generate all Boolean functions.

The lattice viewpoint makes the notion of *precomplete* classes precise. A clone is precomplete if it is maximal among proper clones, meaning that adjoining any operation not already in the clone yields the full clone of all Boolean functions. Post’s theorem identifies the precomplete classes and thereby reduces many questions about Boolean expressibility to membership in one of a small number of maximal closed families. In practice, this means that to show a set of functions is not functionally complete, it suffices to show that it lies inside one of these maximal proper clones. Equivalently, to prove completeness, it is enough to show that the set escapes every precomplete class.

Functional completeness is the property that a set of Boolean functions generates all Boolean functions under composition. In the language of clones, a set is functionally complete exactly when its generated clone is the full clone $\text{Bool}^{\text{Bool}^{<\omega}}$. Classical criteria due to Sheffer show that a single binary connective such as NAND or NOR is functionally complete. More generally, a set of operations is complete if it is not contained in any proper precomplete clone. This gives a decision procedure in principle: test whether the candidate set lies inside one of the known maximal closed classes. The Sheffer criteria are the canonical examples of a basis that avoids all of the standard obstructions at once.

The imported error metric $\varepsilon(r)$ can be read as a comparison score for Boolean representations, combining truth-table Hamming distance with a penalty for variable mismatch. Such a metric is useful when comparing candidate implementations, normal forms, or generated circuits against a target specification. In the present chapter, it plays the role of a practical evaluation tool alongside the algebraic closure theory: one measures how far a proposed function or realization is from a desired Boolean behavior, while the clone-theoretic framework determines whether exact realization is possible by composition. In that sense, $\varepsilon(r)$ is a quantitative companion to the qualitative clone classification.

Algorithmically, clone theory supports several standard tasks. Membership tests ask whether a

given Boolean function belongs to a specified clone such as M , L , D , T_0 , or T_1 . Minimal generating set problems ask for the smallest family of operations whose closure is a target clone or the full Boolean clone. Canonicalization and equivalence procedures often use NPN classes, where functions are identified up to negation of inputs, permutation of variables, and negation of the output. These reductions are especially useful in automated reasoning and circuit synthesis, where many functions differ only by symmetry. In a workflow setting, one typically canonicalizes a candidate, checks clone membership, and then uses the result to infer completeness or incompleteness.

A useful way to read Post’s lattice is as a hierarchy of semantic invariants. Monotone functions preserve the product order on $\{0, 1\}^n$; affine functions are those representable by XORs and constants; self-dual functions satisfy $f(\bar{x}) = \overline{f(x)}$; and the T_0 and T_1 classes preserve the distinguished constants 0 and 1, respectively. Each such property defines a closed class, and intersections of these classes produce further clones. The lattice records exactly how these constraints combine and where they stop being closed under composition. This makes the classification simultaneously semantic and operational: each node corresponds to a recognizable behavior and to a closure condition.

From the standpoint of synthesis, the chapter’s main operational question is whether a target Boolean behavior can be built from a chosen basis. If the basis lies inside a proper clone, then the answer is negative for every function outside that clone. If the basis escapes all proper precomplete clones, then it is complete and can generate every Boolean function. This dichotomy is the Boolean prototype for later chapters, where closure conditions will be studied in richer compositional settings. The Boolean case is especially valuable because the full classification is known and can be used as a reference model for more general closure hierarchies.

The chapter’s organizing principle is that closure is not merely a property of a set of functions, but a design discipline. Once the relevant clone is identified, one can determine which operations are admissible, which are forbidden, and whether a target specification is expressible from a chosen basis. Post’s lattice provides the global map; closure operators and clone membership provide the local mechanics; and functional completeness supplies the synthesis criterion. In this sense, the chapter links algebraic classification to practical circuit design and decision procedures.

In later chapters, these ideas will reappear in more structured settings, including modular composition, symmetry analysis, and algorithmic synthesis. Here they establish the Boolean baseline: the smallest closed families, the maximal proper ones, and the exact boundary between incomplete and complete functional bases. The result is a compact but complete toolkit for reasoning about Boolean expressibility, basis selection, and the algebra of composition.